

Software Architecture and Design Specification

Project: Vehicle Parking System (Project No. 10) – a system for managing parking slots and vehicle tracking

Version: 1.1

Authors: Team 10 – Ramitha R, Pandi Snehitha, Pramiti Ragavendra Udupa

Date: 07-10-2026

Status: Draft – submitted for review

Course: Software Engineering (UE24CS341A) | Semester 5 | Section F | PES University, Bengaluru

Technology: MERN Stack (MongoDB, Express.js, React.js, Node.js)

Team Profile

Sl. No	Name	SRN
1	Ramitha R	PES1UG24CS368
2	Pandi Snehitha	PES1UG24CS315
3	Pramiti Ragavendra Udupa	PES1UG24CS331

Revision History

Version	Date	Author	Change Summary
1.0	05-10-2026	Pramiti Ragavendra Udupa	Initial architecture and design, prepared from the problem statement
1.1	07-10-2026	Pramiti Ragavendra Udupa	Aligned with SRS v1.1 and Software Test Plan v1.1: roles Parking Attendant and Administrator; requirement IDs FR-01...FR-23, NFR-01...NFR-07, SEC-01...SEC-07, BR-01...BR-05; component names from the SRS RTM; fixed 15-minute grace period; new diagrams, API and role matrix, data model and full traceability to STP test cases

Approvals

Role	Name	Signature/Date
Author (Architecture & Design)	Pramiti Ragavendra Udupa	
Reviewer (SRS)	Pandi Snehitha	
Reviewer (QA Lead, Test Plan)	Ramitha R	

Role	Name	Signature/Date
Course Faculty	Ashok Patil	

1. Introduction

1.1 Purpose

This document specifies the software architecture and design of the Vehicle Parking System (VPS), Version 1.1. It turns the requirements of the VPS Software Requirements Specification (SRS v1.1) into components, interfaces, data stores, security controls and interaction flows, and records the reasons for each major architectural decision. Every requirement ID used here is the ID defined in SRS v1.1, and every test-case ID is the ID defined in the Software Test Plan (STP v1.1).

1.2 Scope

The VPS is a web-based system that keeps an accurate, current record of which parking slots are free, which vehicle is parked in which slot, when it entered and left, and how much it owes. It is used by Parking Attendants and Administrators. This document covers the architecture and design of:

- Authentication, user management and role-based access control (FR-01 – FR-05).
- Vehicle registration (FR-06, FR-07).
- Parking slot management and slot availability display (FR-08 – FR-10).
- Vehicle entry with automatic slot allocation (FR-11 – FR-14).
- Vehicle exit, parking fee calculation and slot release (FR-15 – FR-18).
- Hourly fee rate configuration (FR-19).
- Vehicle search, current-location tracking and parking history (FR-20 – FR-22).
- Parking Occupancy Report (FR-23).
- The non-functional, security and business-rule requirements NFR-01 – NFR-07, SEC-01 – SEC-07 and BR-01 – BR-05.

Out of scope (SRS 1.3): online payment, number-plate recognition, hardware gate control, reservations and native mobile applications. Fees are calculated and recorded by the system but collected outside it (cash or counter). The VPS has no interface to any other system (SRS 2.1).

1.3 Audience

The Team 10 developers, the testers who carry out the Software Test Plan, course faculty and reviewers, parking administrators, security reviewers and future maintainers.

1.4 Definitions

Term	Definition
------	------------

Term	Definition
VPS	Vehicle Parking System
Parking Attendant / Administrator	The two user roles of the SRS (2.3); written ATTENDANT and ADMIN in the API
Parking Slot	A uniquely identified parking space (e.g. A-01) with a slot type and a status
Slot Status	Available, Occupied or Unavailable
Parking Record	One visit: vehicle, slot, ticket number, entry time, exit time, fee and status (Active or Completed)
Ticket Number	Unique number given to a parking record at entry
Grace Period	The first 15 minutes of a visit, for which no fee is charged (fixed, SRS FR-16)
Billable Hours	Parking duration minus 15 minutes, rounded up to the next whole hour (0 if the duration is 15 minutes or less)
FR / NFR / SEC / BR	Functional / Non-Functional / Security requirement / Business Rule (SRS v1.1 IDs)
TC	Test case (STP v1.1 IDs, e.g. TC-ENT-01)
MERN	MongoDB, Express.js, React.js, Node.js
SPA	Single Page Application (the React.js front end)
REST / API / JSON	Representational State Transfer / Application Programming Interface / JavaScript Object Notation
JWT	JSON Web Token – the signed session token
RBAC	Role-Based Access Control
ADR	Architecture Decision Record
STRIDE	Threat model: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
TLS / HTTPS	Transport Layer Security / HTTP over TLS
ODM	Object Document Mapper (Mongoose)
SRS / STP / RTM	Software Requirements Specification / Software Test Plan / Requirements Traceability Matrix

2. Document Overview

2.1 How to use this document

Section 3 describes the architecture: goals and constraints, stakeholders, the UML component diagram and component descriptions, the architecture pattern, the technology stack and data stores, risks, traceability to every SRS requirement, the security architecture, the deployment view and the architecture decision records. Section 4 describes the design: module structure and state models, four UML sequence diagrams, the REST API with its role matrix, error handling, logging and monitoring, UX design, and open issues. Section 5 holds the glossary, references and tools. Component names are the names used in the Architecture Reference column of the RTM in SRS Appendix C, and Section 3.8 uses the same test-case IDs as that RTM and STP Section 13.

2.2 Related Documents

- **SRS** – Vehicle Parking System Software Requirements Specification v1.1 (Team 10), including the RTM in Appendix C.
- **STP** – Vehicle Parking System Software Test Plan v1.1 (Team 10), test cases in Appendix B.

- **RTM** – Requirements Traceability Matrix (SRS Appendix C; STP Section 13).
- **Problem statement** – SE Mini-Project Problem Statements, Section F, Project No. 10: "Vehicle Parking System – a system for managing parking slots and vehicle tracking".

3. Architecture

3.1 Goals & Constraints

Goals:

- **Correct allocation:** a slot never has two active vehicles and a vehicle never has two active records, even under concurrent requests (NFR-04, BR-01, BR-03).
- **Durability:** every acknowledged entry or exit survives a server restart (NFR-05).
- **Performance:** vehicle search within 3 s for 95% of requests with 50 concurrent users; entry and exit within 3 s for 95% of operations (NFR-01 – NFR-03).
- **Security:** authentication before every function except login, role-based authorisation on the server, hashed passwords, TLS, input validation, 15-minute idle timeout, secrets outside the code (SEC-01 – SEC-07).
- **Usability and portability:** vehicle entry in at most 5 user actions and 60 s; works in the latest two Chrome, Firefox and Edge versions from 360 px wide (NFR-06, NFR-07).
- **Maintainability:** separate modules (auth, user, vehicle, slot, parking transaction, fee, report) behind service interfaces, so each can be unit-tested (STP Section 5).

Constraints:

- Must be built with the MERN stack and store all data in a persistent MongoDB database (SRS 2.5).
- No interfaces to external systems; no online payment, reservations, number-plate recognition or gate hardware (SRS 1.3, 2.1).
- Grace period fixed at 15 minutes; hourly rates set by the Administrator (SRS 2.6, FR-16, FR-19).
- Timestamps recorded in server time (IST) and stored in UTC; amounts in Indian rupees with two decimal places (SRS 7).
- SRS, SAD and Test Plan must stay traceable through the RTM (SRS 2.5).
- Three-member team working within the mini-project schedule.

3.2 Stakeholders & Concerns

Stakeholder	Primary Concerns
Parking Attendant	Fast and simple vehicle registration, entry and exit (≤ 5 actions); correct slot allocation and fee; finding a vehicle quickly; clear error messages
Administrator	Manage slots, users and hourly rates; accurate occupancy report; only administrators can change these (BR-05)
Developers (Team 10)	Clear module boundaries, a stack the team knows, testable services
Testers	Requirement IDs and test-case IDs that line up; a role matrix to test RBAC (STP TC-SEC-02)

Stakeholder	Primary Concerns
Course faculty / reviewers	IEEE-style documentation, sound architecture rationale, security and traceability coverage

3.3 Component (UML) Diagram

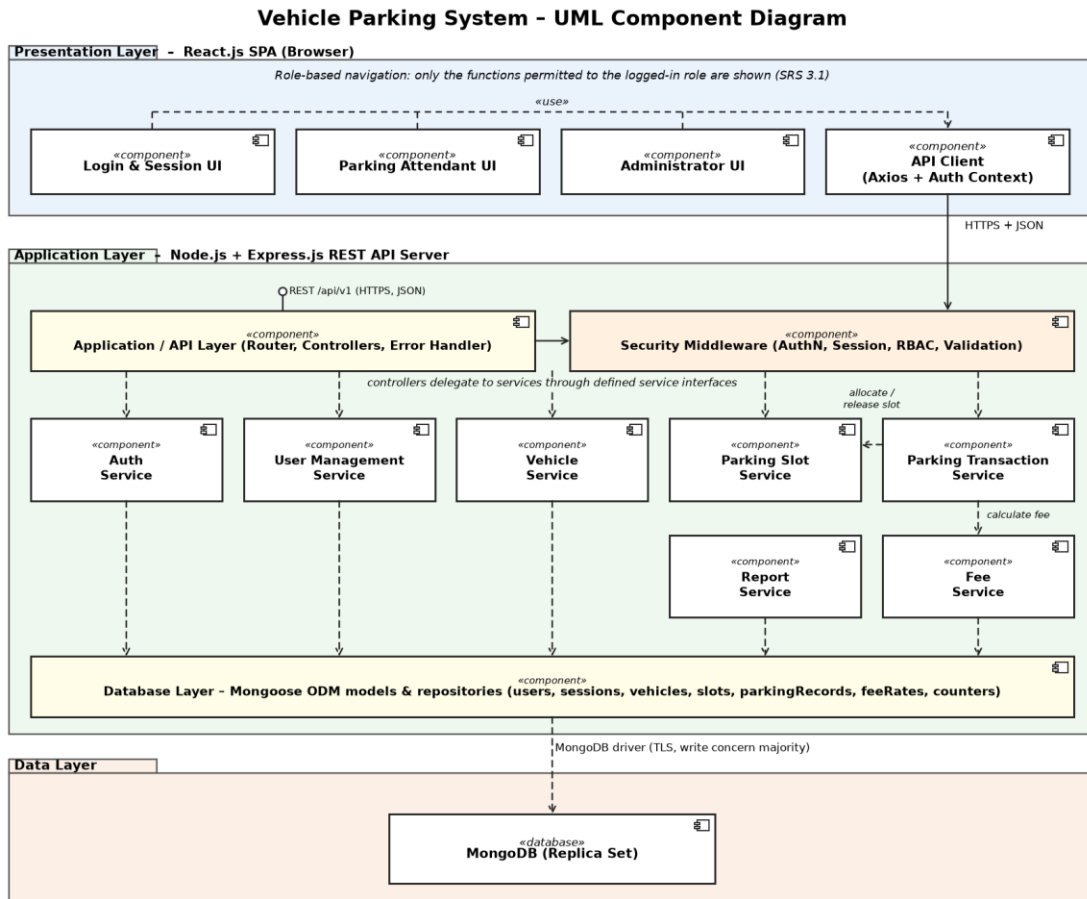


Figure 1: UML component diagram of the Vehicle Parking System (layered, MERN)

3.4 Component Descriptions

Component names match the Architecture Reference column of the SRS RTM (Appendix C).

Component	Responsibility	Requirements
Presentation Layer (React.js SPA)	Login screen, Parking Attendant UI and Administrator UI (screens of SRS 3.1). Shows only the functions allowed for the logged-in role; marks mandatory fields with *; shows user, role and Logout on every screen. Uses the API Client (Axios), which attaches the token and maps error codes to messages.	SRS 3.1, NFR-06, NFR-07
Application / API Layer	Express.js router and controllers for /api/v1; controllers parse the request, call one service and shape the HTTP response; a central error handler returns the standard error format (Section 4.4).	SRS 3.2, 3.3, NFR-01 – NFR-03
Security Middleware	Runs before every controller except login: verifies the JWT and its server-side session (logout, 15-minute idle timeout), checks the role	FR-05, SEC-01, SEC-02, SEC-05, SEC-06

Component	Responsibility	Requirements
	against the role matrix (Section 4.3), and validates and sanitises every input.	
Auth Service	Login with username and password, logout, bcrypt password hashing and checking, session creation, refresh of last activity, and revocation.	FR-01 – FR-03, SEC-03, SEC-06
User Management Service	Administrator creates, updates and deactivates Parking Attendant and Administrator accounts and assigns roles.	FR-04, BR-05
Vehicle Service	Registers vehicles (registration number of 4–15 letters/digits with hyphens and spaces allowed, stored in upper case; vehicle type; optional owner contact) and searches by registration number.	FR-06, FR-07, FR-20, FR-21
Parking Slot Service	Adds slots, sets Unavailable/Available (rejected when Occupied), lists every slot with per-type counts, and provides the atomic allocate/release operations used by the Parking Transaction Service.	FR-08 – FR-10, FR-12, BR-02
Parking Transaction Service	Vehicle entry (check no active record, allocate slot, create record with ticket number), exit (store exit time, get fee, show summary), confirm exit (complete record, release slot), current status and parking history.	FR-11 – FR-18, FR-21, FR-22, NFR-02 – NFR-04, BR-01, BR-03
Fee Service	Stores the hourly rate for each vehicle type and calculates the fee with the fixed 15-minute grace period, using the rate in force when the exit is recorded.	FR-16, FR-17, FR-19, BR-04
Report Service	Parking Occupancy Report: each slot with its status, assigned vehicle and last-updated time.	FR-23
Database Layer	Mongoose schemas, validation, unique and partial-unique indexes, transactions and repositories for all collections (Section 3.6).	NFR-05, SRS 7
MongoDB (Data Layer)	Persistent store run as a 3-member replica set (needed for multi-document transactions), with journaled writes and write concern majority.	NFR-04, NFR-05
Deployment Architecture	Nginx with TLS 1.2+, Node.js under PM2, secrets in environment configuration (Section 3.10).	SEC-04, SEC-07

3.5 Chosen Architecture Pattern and Rationale

Pattern: layered (three-tier client–server) architecture. Inside the back end the layers are Router/Controller → Security Middleware → Service → Database Layer (repository), and each layer calls only the layer below it.

- **Presentation layer:** React.js SPA in the browser; holds no business rules (input hints only).
- **Application layer:** Node.js/Express.js REST API containing the security middleware and the business services (auth, user management, vehicle, parking slot, parking transaction, fee, report).
- **Data layer:** MongoDB accessed only through Mongoose models in the Database Layer.

Rationale:

- Matches the SRS: a React.js front end calling an Express.js REST API that applies the business rules and stores everything in MongoDB (SRS 2.1, 2.5).
- All security checks (authentication, sessions, RBAC, validation) sit in one middleware layer, so every endpoint is protected the same way (SEC-01, SEC-02, SEC-05).
- Each service can be unit-tested with a mocked repository (Jest) and each route with Supertest, as the STP plans (STP Section 5).
- Business rules that must never break (BR-01 – BR-03) are enforced in the service layer and backed by database indexes and transactions.

Alternatives considered and rejected:

- **Microservices:** rejected – one parking facility does not need independent scaling, and distributed transactions would make NFR-04 (no double allocation) much harder.
- **Server-rendered monolith (templates):** rejected – the SRS asks for a React.js front end calling a REST API.
- **Real-time push (WebSockets):** not required by the SRS; the slot view refreshes on load, after every entry/exit, and every 30 s.

3.6 Technology Stack & Data Stores

Layer / Concern	Technology
Front end	React.js 18, Vite, React Router, Axios, Tailwind CSS
Back end	Node.js LTS, Express.js 4, Joi (validation), express-mongo-sanitize, Helmet, express-rate-limit
Database	MongoDB (3-member replica set), Mongoose ODM; separate test database
Security	bcrypt (cost factor 12; SEC-03 requires ≥ 10), jsonwebtoken (HS256), server-side session store, TLS 1.2+ via Nginx
Logging	Winston JSON logs (passwords and tokens never logged)
Testing (per STP)	Jest, Supertest, Cypress, Postman/Newman, Apache JMeter, OWASP ZAP, testssl.sh, gitleaks, npm audit
DevOps	Git and GitHub (team repository, GitHub Issues for defects), PM2, Nginx on Ubuntu 22.04

Data stores (MongoDB collections, from SRS Section 7 and Appendix B):

Collection	Key Fields	Indexes / Notes
users	_id, username, name, passwordHash, role (ATTENDANT ADMIN), active	unique(username)
sessions	jti, userId, createdAt, lastActivityAt, revoked	unique(jti); TTL removes expired sessions
vehicles	_id, regNo (upper case), vehicleType (TWO_WHEELER CAR OTHER), ownerContact (optional), active	unique(regNo)
slots	slotId (e.g. A-01), slotType, status (AVAILABLE OCCUPIED UNAVAILABLE), lastUpdated	unique(slotId); index(slotType, status, slotId)
parkingRecords	ticketNo, vehicleId, slotId, entryTime, exitTime, durationMin, fee, status (ACTIVE COMPLETED), recordedBy	unique(ticketNo); partial-unique(vehicleId) and partial-unique(slotId) where status = ACTIVE (BR-01, BR-03)
feeRates	vehicleType, hourlyRate (₹, ≥ 0), updatedAt, updatedBy	unique(vehicleType)
counters	name, seq	Generates unique ticket numbers, e.g. T-20261007-0042

Fee rule (FR-16, BR-04): duration $d = \text{exitTime} - \text{entryTime}$ in minutes; if $d \leq 15$ the fee is ₹0, otherwise $\text{billableHours} = \text{ceil}((d - 15) \div 60)$ and $\text{fee} = \text{billableHours} \times \text{hourlyRate}$ of the vehicle type. The rate in force when the exit is recorded is used, so a changed rate applies only to exits recorded after the change (FR-19). Money is stored as an integer number of paise and shown with two decimals. With the proposed default Car rate of ₹30/hour: 10 min → ₹0, 15 min → ₹0, 16 min → ₹30, 75 min → ₹30, 76 min → ₹60, 195 min → ₹90; Two-wheeler (₹10/hour) 76 min → ₹20. These are the expected values of STP TC-EXT-03.

3.7 Risks & Mitigations

#	Risk	Mitigation
R1	Two attendants allocate the same slot at the same moment (NFR-04)	Atomic findOneAndUpdate on slot status inside a transaction, plus a partial-unique index on Active records per slot; the losing request gets 409 and is asked to retry (TC-CONC-01)
R2	Acknowledged entry or exit lost on a crash (NFR-05)	Replica set with journaled, majority-acknowledged writes; the API responds only after commit (TC-REL-01)
R3	Default hourly rates not yet confirmed (SRS 2.6)	Rates are data in feeRates, editable by the Administrator; tests use data-driven expected values (STP risk table)
R4	Password guessing on the login endpoint	Generic error message (FR-02), bcrypt hashing (SEC-03), request rate limiting on /auth/login
R5	Attendant performs an administrator function	Server-side role matrix returns 403 even when the UI hides the control (FR-05, SEC-02, TC-AUTH-05)
R6	Secrets leak through the repository	Secrets only in .env (not committed); gitleaks scan (SEC-07, TC-SEC-07)

3.8 Traceability to Requirements

Each SRS v1.1 requirement is mapped to the architecture component that realises it (Architecture Reference), the section of this document that designs it (Design Reference), and the STP v1.1 test case(s) that verify it. The test-case IDs are the same as in SRS Appendix C and STP Section 13.

Req. ID	Requirement (summary)	Architecture Reference	Design Reference	Test Case ID
FR-01	Login, issue session token	Auth Service	4.2 SD-3; 4.3 (a)	TC-AUTH-01
FR-02	Generic invalid-credentials message	Auth Service	4.2 SD-3; 4.3 (a)	TC-AUTH-02
FR-03	Logout; token no longer accepted	Auth Service	4.2 SD-3; 3.9	TC-AUTH-03
FR-04	Admin manages accounts and roles	User Management Service	4.3 role matrix	TC-AUTH-04
FR-05	403 for functions outside role	Security Middleware	3.9; 4.3 role matrix	TC-AUTH-05
FR-06	Register vehicle; format; upper case	Vehicle Service	4.3 (b)	TC-VEH-01, TC-VEH-03
FR-07	Reject empty, invalid or duplicate	Vehicle Service	4.3 (b); 4.4	TC-VEH-02, TC-VEH-03
FR-08	Admin adds a slot	Parking Slot Service	4.3 (c)	TC-SLOT-01
FR-09	Unavailable / Available, not if Occupied	Parking Slot Service	4.1 state model; 4.3	TC-SLOT-02
FR-10	Display slots and counts per type	Parking Slot Service	4.3 (c); 4.5	TC-SLOT-03
FR-11	Record entry; reject if already parked	Parking Transaction Service	4.2 SD-1; 4.3 (d)	TC-ENT-01, TC-ENT-02
FR-12	Allocate lowest matching slot / manual choice	Parking Transaction Service	4.2 SD-1	TC-ENT-01, TC-ENT-04
FR-13	Refuse when no matching slot	Parking Transaction Service	4.2 SD-1 (alt)	TC-ENT-03
FR-14	Atomic record + ticket + slot Occupied	Parking Transaction Service	4.2 SD-1; 3.6	TC-ENT-01
FR-15	Record exit time; reject without active record	Parking Transaction Service	4.2 SD-2; 4.3 (e)	TC-EXT-01, TC-EXT-02
FR-16	Fee with fixed 15-min grace, rounded up	Fee Service	3.6 fee rule; 4.2 SD-2	TC-EXT-03, TC-FEE-02
FR-17	Show fee summary before confirm	Fee Service	4.2 SD-2; 4.5	TC-EXT-01
FR-18	Complete record, release slot	Parking Transaction Service	4.2 SD-2	TC-EXT-01
FR-19	Hourly rate per type; applies to later exits	Fee Service	4.3 (f); 3.6 fee rule	TC-FEE-01, TC-FEE-02
FR-20	Search vehicle by registration number	Vehicle Service	4.2 SD-4; 4.3 (g)	TC-SRCH-01
FR-21	Current status Parked / Not Parked	Vehicle Service / Parking Transaction Service	4.2 SD-4	TC-SRCH-02
FR-22	Parking history, newest first	Parking Transaction Service	4.2 SD-4	TC-SRCH-03
FR-23	Parking Occupancy Report	Report Service	4.3	TC-REP-01
NFR-01	Search ≤ 3 s, 95%, 50 users	Application/API Layer	3.6 indexes; 4.4	TC-PERF-01

Req. ID	Requirement (summary)	Architecture Reference	Design Reference	Test Case ID
NFR-02	Entry with allocation ≤ 3 s, 95%	Parking Transaction Service	4.2 SD-1; 4.4	TC-PERF-02
NFR-03	Exit with fee ≤ 3 s, 95%	Parking Transaction Service	4.2 SD-2; 4.4	TC-PERF-02
NFR-04	No double allocation under concurrency	Parking Transaction Service	4.2 SD-1 (alt); 3.11 ADR-02	TC-CONC-01
NFR-05	Acknowledged operations persisted	Database Layer	3.10; 3.11 ADR-02	TC-REL-01
NFR-06	Entry in ≤ 5 actions, ≤ 60 s	Presentation Layer	4.5	TC-USAB-01
NFR-07	Latest two Chrome/Firefox/Edge, ≥ 360 px	Presentation Layer	4.5	TC-PORT-01
SEC-01	Authenticate before every function except login	Security Middleware	3.9; 4.3 role matrix	TC-AUTH-01, TC-SEC-01
SEC-02	Server-side RBAC on every endpoint	Security Middleware	3.9; 4.3 role matrix	TC-AUTH-05, TC-SEC-02
SEC-03	bcrypt hashes, cost ≥ 10 ; no passwords in logs	Auth Service	3.9; 4.4	TC-SEC-03
SEC-04	HTTPS with TLS 1.2+	Deployment Architecture	3.9; 3.10	TC-SEC-04
SEC-05	Server-side input validation	Security Middleware	3.9; 4.4	TC-SEC-05
SEC-06	End session after 15 min inactivity	Auth Service	4.2 SD-3; 3.9	TC-SEC-06
SEC-07	Secrets in environment configuration	Deployment Architecture	3.9; 3.10	TC-SEC-07
BR-01	One active record per slot	Parking Transaction Service	3.6 indexes	TC-CONC-01
BR-02	Allocate only Available, type-matching slots	Parking Slot Service	4.2 SD-1	TC-ENT-01
BR-03	One active record per vehicle; exit needs one	Parking Transaction Service	3.6 indexes; 4.2 SD-2	TC-ENT-02, TC-EXT-02
BR-04	Fees only from admin rates + fixed grace	Fee Service	3.6 fee rule	TC-EXT-03
BR-05	Only Admin manages slots, users, rates, reports	Security Middleware	4.3 role matrix	TC-AUTH-05, TC-REP-01

Coverage: all 42 requirement items of SRS v1.1 (23 FR, 7 NFR, 7 SEC, 5 BR) are mapped to an architecture component, a design section and at least one STP v1.1 test case.

3.9 Security Architecture

Security objectives (SRS 6.3): SO-1 Confidentiality of user, vehicle and parking data; SO-2 Integrity of slot status, parking records and fee rates; SO-3 Access control – privileged operations only for users with the required role.

Security controls by layer:

- **Authentication (SEC-01):** every endpoint except POST /auth/login requires a valid token and session; a missing or malformed token gets 401.
- **Password storage (SEC-03):** passwords are stored only as salted bcrypt hashes (cost 12); request bodies of /auth routes are never logged.
- **Sessions (FR-03, SEC-06):** the JWT carries a session id (jti). Each request checks the session record: if it is revoked (logout) or idle for more than 15 minutes the request gets 401; otherwise lastActivityAt is updated. The SPA keeps the token in memory, not in localStorage.
- **Authorisation (FR-05, SEC-02, BR-05):** the role matrix in Section 4.3 is enforced on the server for every endpoint; hiding a control in the UI is never relied on. A forbidden call returns 403 with no protected data.
- **Input validation (SEC-05):** Joi schemas check type, length and format of every field; express-mongo-sanitize rejects keys that start with \$ or contain dots; text containing script content is rejected with 400; React escapes all output.

- **Transport (SEC-04):** all client–server traffic over HTTPS with TLS 1.2 or higher, terminated at Nginx; plain HTTP is redirected to HTTPS; MongoDB connections use TLS and authentication.
- **Secrets (SEC-07):** database URI and token-signing secret are read from environment variables (.env, excluded from Git); the server refuses to start without them.
- **Error responses (SRS 3.3):** no stack traces or database details in any error.

Threat modelling (STRIDE):

Threat	Example in VPS	Mitigation
Spoofing	Guessing an attendant's password; reusing a token after logout	bcrypt (SEC-03), generic error (FR-02), login rate limit, session revocation and idle timeout (FR-03, SEC-06)
Tampering	Sending a lower fee or a different exit time from the browser	Fee and timestamps computed on the server from stored rates and server time (FR-15, FR-16); input validation (SEC-05)
Repudiation	Disagreement about who recorded an entry or exit	recordedBy and server timestamps stored on every parking record; Completed records are not editable through the API
Information disclosure	Reading data without logging in; traffic sniffing	Authentication on all endpoints (SEC-01), TLS (SEC-04), minimal error messages
Denial of service	Flooding login or search	Rate limiting, request size limits, Nginx connection limits, indexed queries
Elevation of privilege	Attendant calls POST /slots, PUT /fee-rates, POST /users or GET /reports/occupancy	Server-side RBAC returns 403 (FR-05, SEC-02, TC-AUTH-05); role read only from the verified session

3.10 Deployment View

Vehicle Parking System - Deployment View

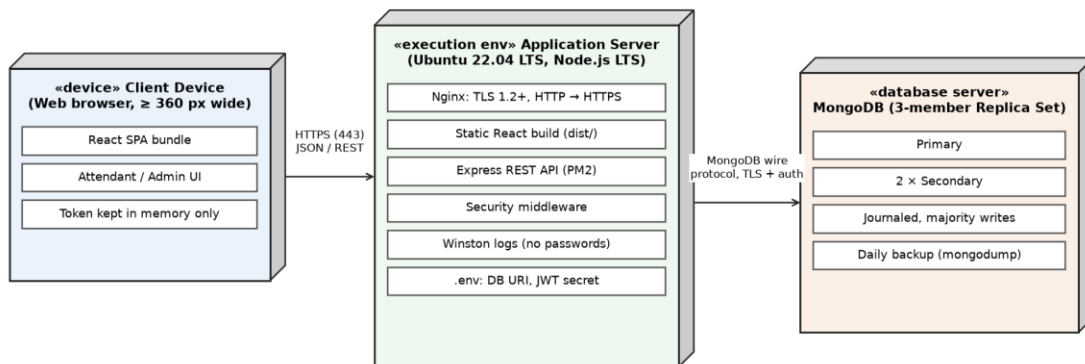


Figure 2: Deployment view of the Vehicle Parking System

Nginx serves the React build and forwards /api/v1 requests to the Express API, which runs under PM2 (restarted automatically on failure). MongoDB runs as a 3-member replica set so that multi-document transactions are available and acknowledged writes survive a restart (NFR-04,

NFR-05). Each team member develops locally with a single-node replica set; tests use a separate test database (STP Section 6).

3.11 Architecture Decision Records (ADRs)

ADR	Decision	Status / Consequence
ADR-01	Layered MERN architecture with separate service modules	Accepted – matches SRS 2.1/2.5; simple to deploy and to unit-test
ADR-02	Entry and exit-confirm run in MongoDB transactions, backed by partial-unique indexes on Active records	Accepted – guarantees NFR-04, BR-01, BR-03 even if application logic has a bug; needs a replica set
ADR-03	JWT plus server-side session record	Accepted – lets logout invalidate the token (FR-03) and enforces the 15-minute idle timeout (SEC-06)
ADR-04	Fees always computed on the server from stored rates at exit time	Accepted – prevents tampering; rate changes need no code change (FR-16, FR-19, BR-04)
ADR-05	Slot view refreshes on demand and every 30 s instead of WebSockets	Accepted – no real-time requirement in the SRS; fewer moving parts

4. Design

4.1 Design Overview

The back end is organised by module. Every module follows the same layering:

```
server/src/  
  routes/      auth, users, vehicles, slots, parking,  
               fee-rates, reports (*.routes.js)  
  middleware/  authenticate (JWT + session), authorize(roles),  
               validate(schema), errorHandler  
  controllers/ thin: parse request -> call service -> HTTP response  
  services/    AuthService, UserService, VehicleService,  
               SlotService, ParkingService, FeeService, ReportService  
  models/      User, Session, Vehicle, Slot, ParkingRecord,  
               FeeRate, Counter (Mongoose)  
  config/      env.js (reads .env, fails fast if missing)  
client/src/  
  pages/login, pages/attendant, pages/admin, components/,  
  api/ (Axios client), context/AuthContext
```

Design rules: controllers contain no business logic; services depend on repositories that can be mocked in unit tests; all times come from the server clock; money is stored as integer paise.

State models:

- **Slot (FR-08, FR-09, FR-14, FR-18):** AVAILABLE → OCCUPIED (entry) → AVAILABLE (exit confirmed); AVAILABLE ↔ UNAVAILABLE (Administrator). An OCCUPIED slot cannot be set UNAVAILABLE.
- **Parking record (FR-14 – FR-18):** ACTIVE (created at entry; exitTime and fee set when exit is recorded) → COMPLETED (exit confirmed). If the attendant cancels before confirming,

exitTime and fee are cleared and the record stays ACTIVE. The API offers no update or delete for COMPLETED records.

- **User account (FR-04):** active ↔ deactivated (Administrator); a deactivated user cannot log in (TC-AUTH-04).

4.2 UML Sequence Diagrams

Four sequence diagrams cover the main flows: vehicle entry with slot allocation (SD-1), vehicle exit with fee calculation (SD-2), login, session timeout and logout (SD-3), and vehicle search and tracking (SD-4).

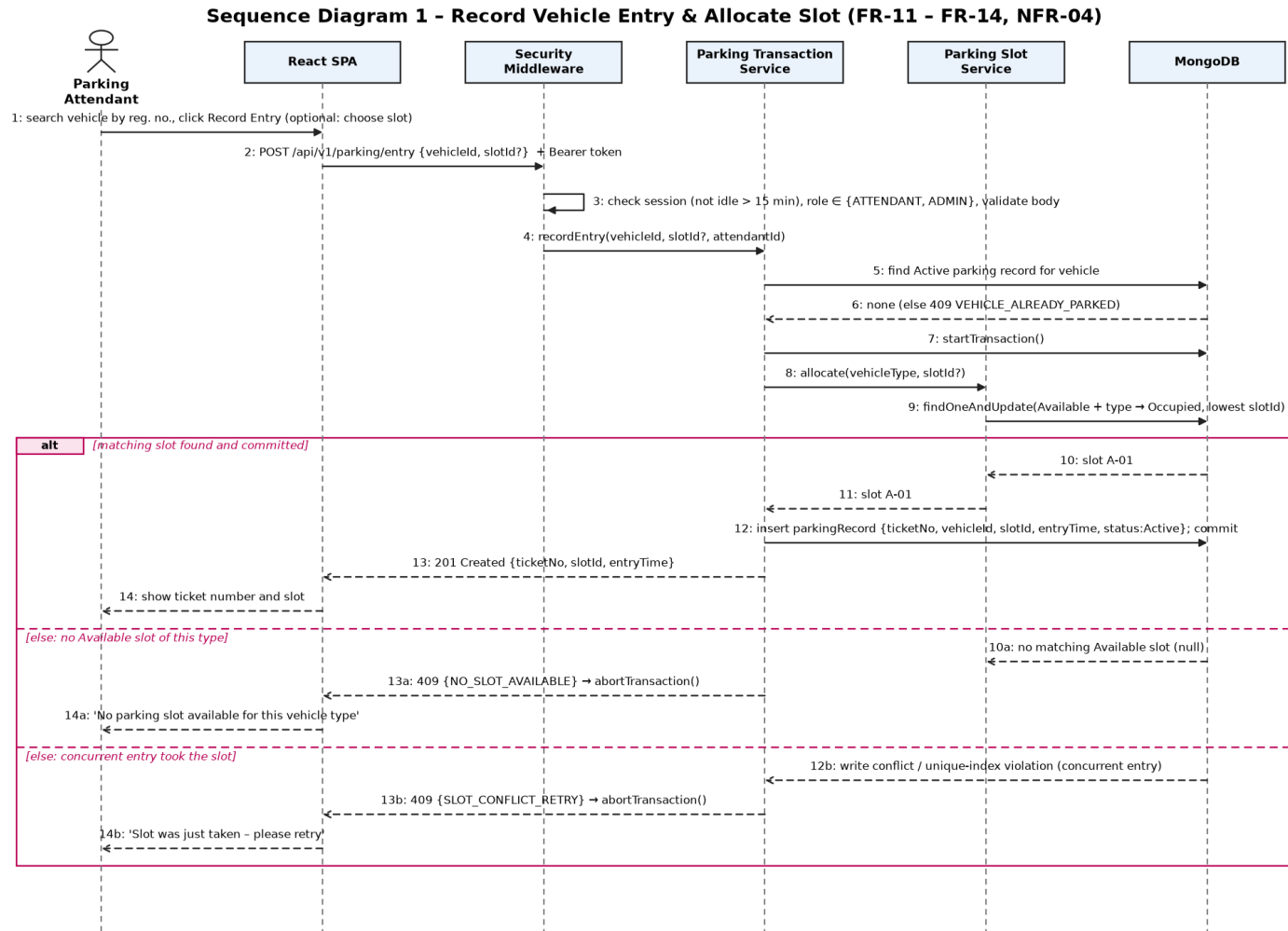


Figure 3: SD-1 – record vehicle entry and allocate a slot

SD-1: the attendant records entry for a registered vehicle. The Parking Transaction Service rejects the entry if the vehicle already has an Active record (FR-11). Inside one transaction the Parking Slot Service atomically switches the lowest-numbered Available slot of the right type to Occupied (or the slot the attendant chose, FR-12), and a parking record with a unique ticket number is inserted (FR-14). If no slot matches, the entry is refused (FR-13). If a concurrent entry wins the same slot, the transaction aborts and the attendant is asked to retry (NFR-04).

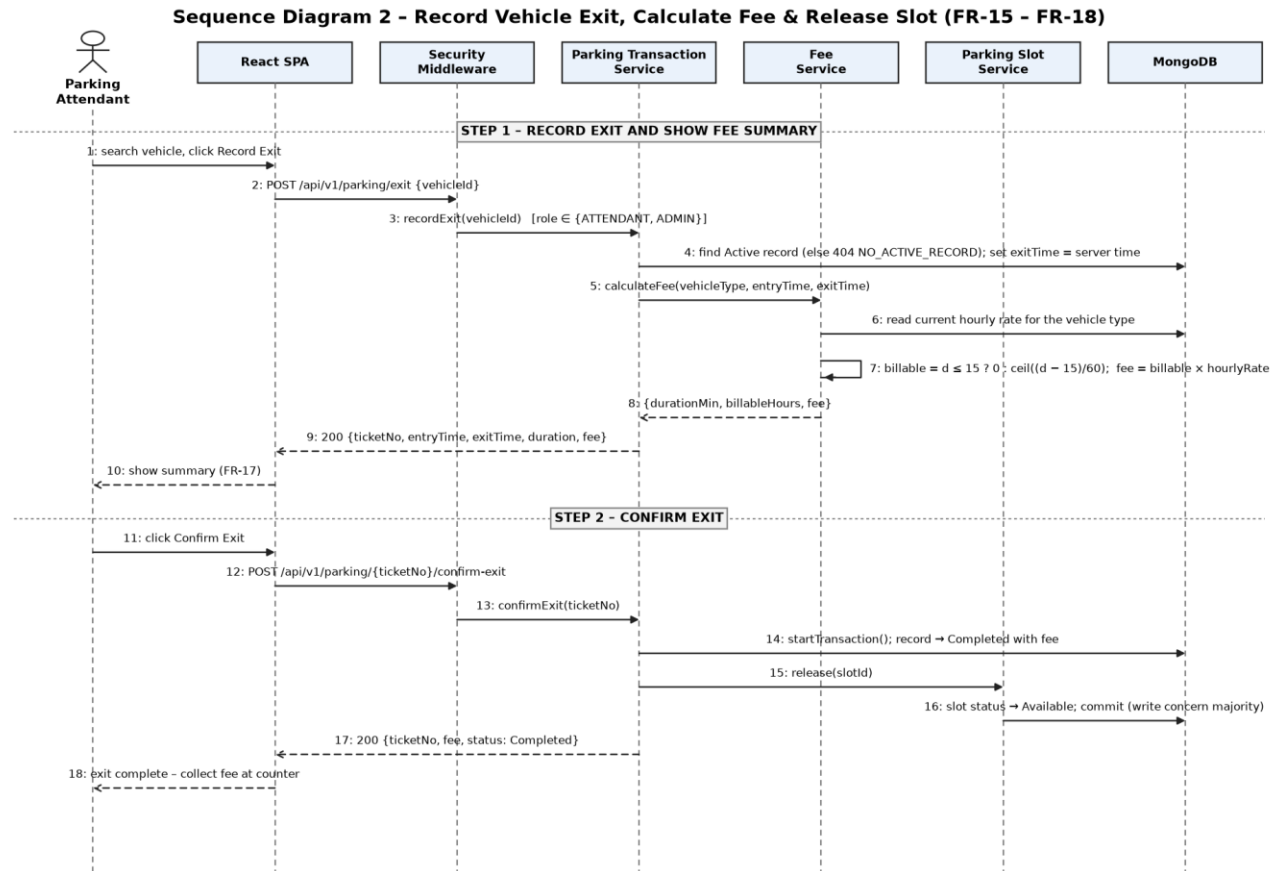


Figure 4: SD-2 – record vehicle exit, calculate fee and release the slot

SD-2: step 1 stores the exit time and computes the fee from the current hourly rate with the fixed 15-minute grace period, and the summary is shown before anything is final (FR-15 – FR-17). Step 2, on confirmation, marks the record Completed with the fee and sets the slot Available in one transaction (FR-18).

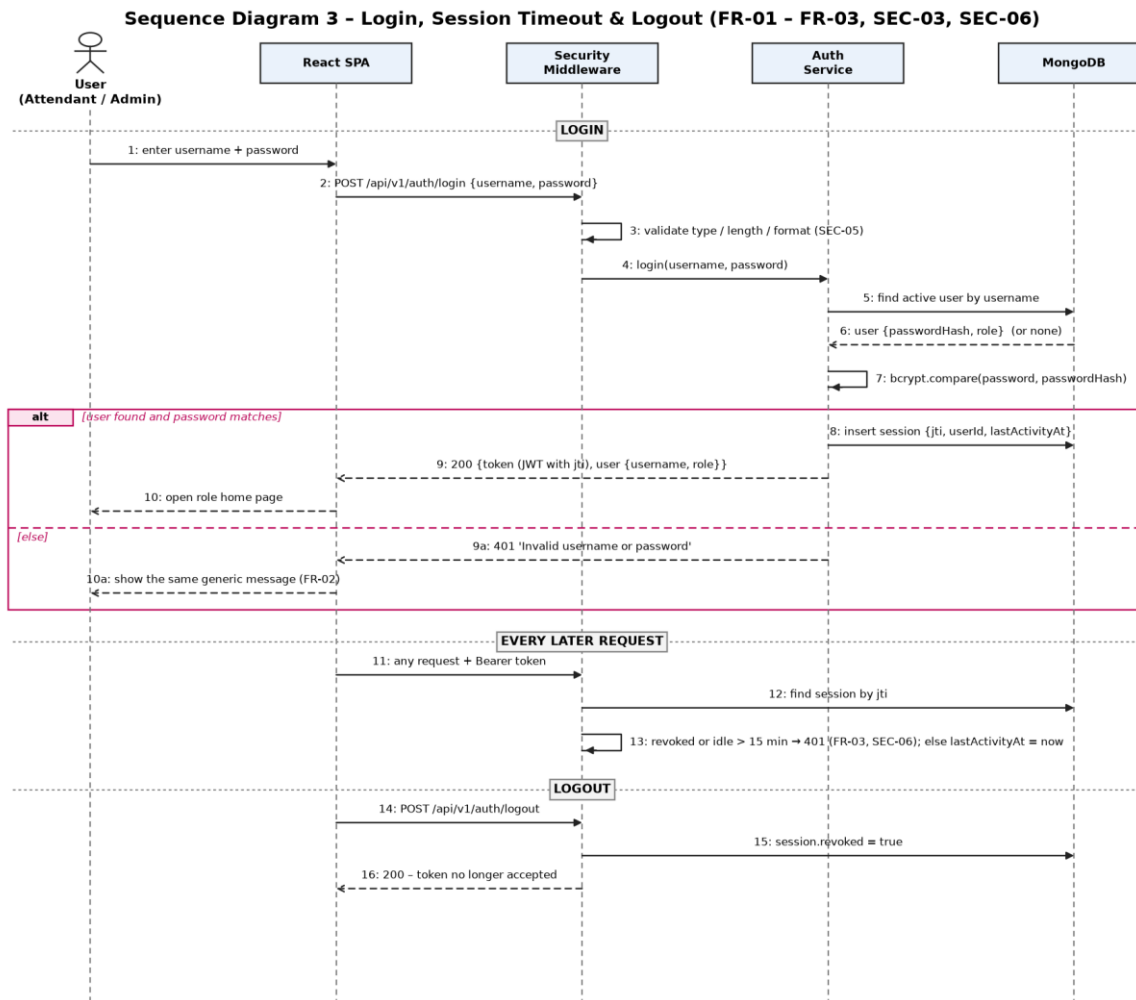


Figure 5: SD-3 – login, session timeout and logout

SD-3: input is validated and the password is checked with bcrypt. On success a server-side session is created and a JWT carrying its id is returned (FR-01); otherwise the generic message "Invalid username or password" is returned (FR-02). Every later request checks the session: revoked (after logout, FR-03) or idle for more than 15 minutes (SEC-06) gives 401.

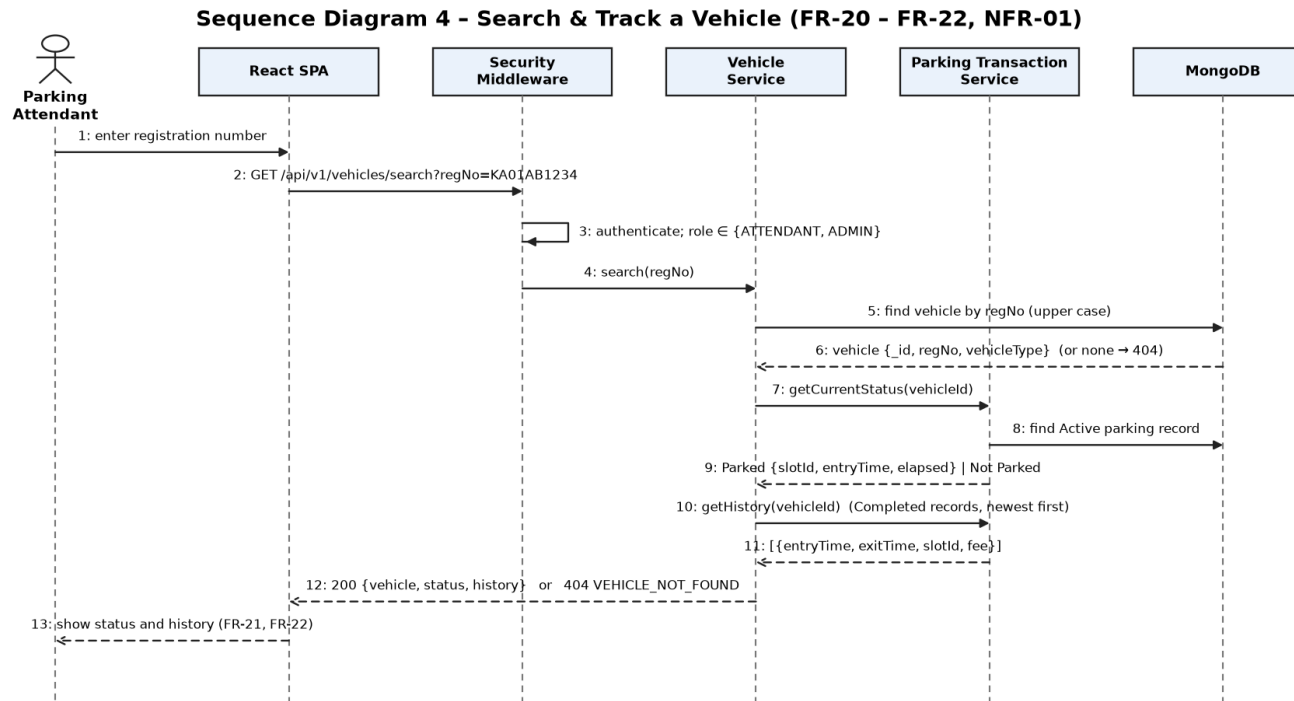


Figure 6: SD-4 – search and track a vehicle

SD-4: the attendant or administrator searches by registration number (FR-20). The response gives the current status – Parked with slot, entry time and elapsed time, or Not Parked (FR-21) – and the history of Completed records, newest first (FR-22). An unregistered number returns "Vehicle not found" (TC-SRCH-01).

4.3 API Design

Base URL `https://<host>/api/v1`. JSON requests and responses; every endpoint except login needs the header `Authorization: Bearer <token>`. Status codes follow SRS 3.3 (200, 201, 400, 401, 403, 404, 409, 500).

Endpoint and role matrix (enforced on the server; this is the role matrix used by STP TC-SEC-02). Y = allowed; – = 403 Forbidden; requests without a valid session get 401.

Component	Endpoint	Method	No login	Attendant	Admin	Req.
Auth	/auth/login	POST	Y	Y	Y	FR-01, FR-02
Auth	/auth/logout	POST	–	Y	Y	FR-03
User Mgmt	/users	GET, POST	–	–	Y	FR-04
User Mgmt	/users/{userId}	PUT	–	–	Y	FR-04
User Mgmt	/users/{userId}/status	PATCH	–	–	Y	FR-04
Vehicle	/vehicles	POST	–	Y	Y	FR-06, FR-07
Vehicle	/vehicles/search?regNo=	GET	–	Y	Y	FR-20, FR-21
Parking	/vehicles/{vehicleId}/history	GET	–	Y	Y	FR-22
Slot	/slots, /slots/summary	GET	–	Y	Y	FR-10
Slot	/slots	POST	–	–	Y	FR-08
Slot	/slots/{slotId}/status	PATCH	–	–	Y	FR-09
Parking	/parking/entry	POST	–	Y	Y	FR-11 – FR-14
Parking	/parking/exit	POST	–	Y	Y	FR-15 – FR-17
Parking	/parking/{ticketNo}/confirm-exit	POST	–	Y	Y	FR-18
Parking	/parking/{ticketNo}/cancel-exit	POST	–	Y	Y	FR-17
Fee	/fee-rates	GET	–	Y	Y	FR-19
Fee	/fee-rates	PUT	–	–	Y	FR-19
Report	/reports/occupancy	GET	–	–	Y	FR-23
Health	/health	GET	Y	Y	Y	4.4 (no data)

Detailed interface definitions:

(a) Auth Service – Login

Endpoint: `/api/v1/auth/login` Method: POST Role: none
Request: { "username": "att1", "password": "*****" }
Response: 200 OK
{ "token": "<JWT with jti>",
"user": { "username": "att1", "role": "ATTENDANT" } }
Errors: 400 VALIDATION_ERROR (missing / over-long / wrong type /
database operators)
401 INVALID_CREDENTIALS "Invalid username or password"
(same for unknown user, wrong password, deactivated user)

(b) Vehicle Service – Register Vehicle

Endpoint: `/api/v1/vehicles` Method: POST
Role: ATTENDANT, ADMIN
Request: { "regNo": "ka01ab1234", "vehicleType": "CAR",
"ownerContact": "9876543210" } (ownerContact optional)
Response: 201 Created
{ "vehicleId": "6703cd...", "regNo": "KA01AB1234",
"vehicleType": "CAR" }
Errors: 400 VALIDATION_ERROR { "field": "vehicleType" } (empty /
not TWO_WHEELER | CAR | OTHER; regNo not 4-15 chars of

```
letters, digits, hyphen, space)
409 DUPLICATE_REG_NO { "field": "regNo" }
```

(c) Parking Slot Service – Add Slot / Slot Availability

```
Endpoint: /api/v1/slots          Method: POST      Role: ADMIN
Request:  { "slotId": "A-01", "slotType": "CAR" }
Response: 201 Created { "slotId": "A-01", "slotType": "CAR",
                        "status": "AVAILABLE" }
Errors:   400 VALIDATION_ERROR   409 DUPLICATE_SLOT_ID   403 FORBIDDEN

Endpoint: /api/v1/slots/summary   Method: GET      Role: ATTENDANT, ADMIN
Response: 200 OK { "byType": [
  { "slotType": "CAR", "available": 6, "occupied": 3 },
  { "slotType": "TWO_WHEELER", "available": 5, "occupied": 0 } ] }
```

(d) Parking Transaction Service – Record Entry

```
Endpoint: /api/v1/parking/entry   Method: POST
Role:     ATTENDANT, ADMIN
Request:  { "vehicleId": "6703cd...", "slotId": "A-05" } (slotId optional)
Response: 201 Created
  { "ticketNo": "T-20261007-0042", "slotId": "A-01",
    "entryTime": "2026-10-07T09:04:12+05:30" }
Errors:   404 VEHICLE_NOT_FOUND
          409 VEHICLE_ALREADY_PARKED
          409 NO_SLOT_AVAILABLE "No parking slot available for this
                                vehicle type"
          409 SLOT_NOT_AVAILABLE (chosen slot busy or wrong type)
          409 SLOT_CONFLICT_RETRY (concurrent entry; please retry)
```

(e) Parking Transaction Service – Record Exit and Confirm

```
Endpoint: /api/v1/parking/exit     Method: POST
Role:     ATTENDANT, ADMIN
Request:  { "vehicleId": "6703cd..." }
Response: 200 OK
  { "ticketNo": "T-20261007-0042", "slotId": "A-01",
    "entryTime": "2026-10-07T09:04:12+05:30",
    "exitTime": "2026-10-07T10:34:12+05:30",
    "durationMin": 90, "billableHours": 2,
    "hourlyRate": "30.00", "fee": "60.00" }
Errors:   404 NO_ACTIVE_RECORD

Endpoint: /api/v1/parking/{ticketNo}/confirm-exit Method: POST
Response: 200 OK { "ticketNo": "...", "status": "COMPLETED",
                  "fee": "60.00", "slotStatus": "AVAILABLE" }
Errors:   404 TICKET_NOT_FOUND   409 ALREADY_COMPLETED
```

(f) Fee Service – Update Hourly Rates

```
Endpoint: /api/v1/fee-rates           Method: PUT           Role: ADMIN
Request:  { "rates": [ { "vehicleType": "TWO_WHEELER", "hourlyRate": 10 },
                      { "vehicleType": "CAR",           "hourlyRate": 30 },
                      { "vehicleType": "OTHER",         "hourlyRate": 50 } ] }
Response: 200 OK (stored rates; used for exits recorded from now on)
Errors:   400 VALIDATION_ERROR (rate < 0 or not a number;
                        stored rate is left unchanged)
          403 FORBIDDEN
```

(g) Vehicle Service – Search and Track

```
Endpoint: /api/v1/vehicles/search?regNo=KA01AB1234    Method: GET
Role:      ATTENDANT, ADMIN
Response: 200 OK
    { "regNo": "KA01AB1234", "vehicleType": "CAR",
      "status": "PARKED", "slotId": "A-01",
      "entryTime": "2026-10-07T09:04:12+05:30",
      "elapsedMin": 42 }
    or { ..., "status": "NOT_PARKED" }
Errors: 404 VEHICLE NOT FOUND "Vehicle not found"
```

4.4 Error Handling, Logging & Monitoring

Error handling:

- Services throw typed errors (code, HTTP status, message); one Express error handler turns them into the standard body: { "error": { "code": "NO_SLOT_AVAILABLE", "message": "No parking slot available for this vehicle type", "field": null } }.
- Validation errors name the offending field so the UI can show the message next to it (FR-07, SRS 3.1).
- Unexpected errors return 500 with a generic message; stack traces and database details are never sent to the client (SRS 3.3).
- Entry and exit-confirm run in transactions and are rolled back on any failure, so slot status and records stay consistent.
- Front end: the Axios client shows the message next to the field or as a banner; a 401 sends the user to the Login page.

Logging:

- Winston JSON application logs with time, request id, user, route, status and duration.
- Passwords and tokens are never written to logs (SEC-03, checked by TC-SEC-03).

Monitoring:

- GET /api/v1/health reports API and database status without any data; PM2 restarts the API on crash.

- Response-time logging per route gives the 95th-percentile figures needed for NFR-01 – NFR-03 (measured with JMeter in TC-PERF-01, TC-PERF-02).
- Indexes on regNo, slotId, slot type/status and Active records keep search, entry and exit fast (NFR-01 – NFR-03).

4.5 UX Design

- **Screens (SRS 3.1):** Login, Slot Availability, Vehicle Registration and Search, Vehicle Entry, Vehicle Exit with fee summary, Parking History, Slot Management, User Management, Fee Rates, Occupancy Report. Menus show only the screens allowed for the role.
- **Every screen:** logged-in username, role and a Logout control; mandatory fields marked with *; error messages shown next to the field.
- **Slot status:** text plus colour – Available (green), Occupied (red), Unavailable (grey) – with Available and Occupied counts per slot type (FR-10).
- **Fast entry (NFR-06):** type registration number → Search → Record Entry → Confirm: 4 actions; the suggested slot is pre-selected.
- **Exit (FR-17):** the summary shows entry time, exit time, duration and fee before Confirm Exit; Cancel leaves the record Active.
- **Responsive (NFR-07):** layout works from 360 px wide on the latest two versions of Chrome, Firefox and Edge, with no horizontal scrolling.

4.6 Open Issues & Next Steps

- Default hourly rates (SRS 2.6) to be confirmed by the reviewer before system testing.
- Fill in the Code File Reference column of the RTM (SRS Appendix C) as implementation progresses.
- Possible later versions: number-plate recognition, online payment, reservations, mobile app (all out of scope for this version).

5. Appendices

5.1 Glossary:

See Section 1.4 and SRS Appendix A (Vehicle, Parking Slot, Slot Status, Parking Record, Vehicle Tracking, Grace Period, Parking Fee, Parking Attendant, Administrator).

5.2 References:

- Team 10 – Vehicle Parking System SRS v1.1 and Software Test Plan v1.1.
- PES University, SE (UE24CS341A) Mini-Project Problem Statements – Section F; Mini-Project Deliverables Part-1; Project Guidelines; SAD template.
- ISO/IEC/IEEE 42010:2022 – Architecture description; IEEE Std 1016-2009 – Software Design Descriptions.
- ISO/IEC/IEEE 29148:2018 – Requirements engineering; IEEE Std 829-2008 – Software test documentation.

- OWASP Top 10 and OWASP Application Security Verification Standard; NIST SP 800-160.

5.3 Tools:

Python/matplotlib (UML diagrams in docs/diagrams), draw.io / PlantUML, Swagger / OpenAPI 3.0, Postman, VS Code, Git & GitHub, MongoDB Compass, Jest, JMeter.